

kithbuysheet

Vendor catalog → filled BUYSHEET, in one pipeline

Operational overview

What it does

A pipeline that turns a vendor's wholesale shoe catalog (PDF) into a filled BUYSHEET Excel file. The goal: replace the manual data-entry work of typing every SKU, color, price, and product description from a vendor's PDF into our buy-sheet template.

On a typical 30-page catalog, end-to-end cost is roughly \$3–\$5 in model calls, and re-running the same catalog later is nearly free thanks to on-disk caching. Single-vendor catalog runs are production-quality today. Multi-vendor merging is built but still in beta.

Bottom line

Single-catalog runs are reliable and cheap. Multi-catalog merging works but needs more real-world testing before we trust it on vendor disagreements. Anything the system isn't sure about gets flagged yellow and listed on a REVIEW sheet — no silent failures.

How it works, in plain terms

The pipeline runs six steps in order. Each step writes its output to disk so the next step can pick it up — and so a crash anywhere is recoverable without redoing the expensive parts.

1. Render

Splits the PDF into one image per page at high resolution. Cached.

2. Classify

A cheap model (Sonnet) looks at each page image and tags it: brand cover, category header, product page, or other. Cached.

3. Extract

For every product page, an expensive model (Opus) reads the image and pulls out the structured product rows: SKU, description, color, cost, retail, intro date, gender. Cached.

4. Verify

Each field the model claimed to read is checked, for free, against the PDF's underlying text layer using substring matching. Anything that fails verification gets flagged.

5. Map vocabulary

Free-text values ("core black," "low-top," etc.) are mapped to the BUYSHEET's dropdown values by Sonnet. This cache is shared across vendors — by the third catalog, most lookups are free.

6. Build

Writes the final BUYSHEET.xlsx, applies yellow highlights to any flagged rows, and appends a REVIEW sheet so the buyer knows exactly where to look.

Why two models, not one

Every model call is shaped to play to a model's strengths, so we never overpay.

Role	Model	What it does	Why this one
Reader	Opus (vision)	Reads each product page and extracts product rows	High accuracy where mistakes are expensive; runs once per page max
Judge	Sonnet	Classifies pages, maps dropdowns, picks image-detection density, matches photos to SKUs	Cheap and fast; many calls, easy to cache
Verifier	None	Substring-matches the reader's output against the PDF's text layer	Free; replaces what used to be a paid model call

Photos in the buy-sheet (optional)

On single-catalog runs, the pipeline can also crop each shoe's photo from the catalog and embed it as a thumbnail next to the SKU row in column A of the buy-sheet. This step is automatic when the optional photo-detection dependencies are installed, and it's silently skipped otherwise — the rest of the pipeline is unaffected.

How it works: Sonnet looks at the page and decides whether shoes are arranged loosely or densely. A computer-vision detector (YOLO-World) draws boxes around every shoe at the appropriate resolution. Then Sonnet looks at the page with numbered boxes overlaid and matches each box to its SKU. Marketing or lifestyle shots are skipped on purpose.

Measured photo-matching coverage

Catalog	Photos matched to SKUs	Coverage
Converse	15 of 15	100%
Salomon	147 of 150	98.0%
adidas	314 of 333	94.3%

How problems surface

The buyer should never have to read JSON or read logs to find issues. The output Excel file makes them obvious in three ways:

- **Yellow highlight on the SKU# cell** of any row whose source page was flagged — extraction error, a product page that returned zero products, missing text layer, any unverified field, or the photo matcher couldn't place a thumbnail.
- **A REVIEW sheet** is appended to every output file, with one row per flagged page and a summary of what couldn't be verified.
- **An analysis report** (per catalog) lists fill rate per column, how many SKUs were dropped at each stage, and the dollar spend per model step.

What's production-ready vs. in review

Capability	Status
Single-catalog PDF → BUYSHEET	Production-ready. Validated on Converse, Salomon, adidas.
Photo embedding in column A	Production-ready on single-catalog runs. 94–100% coverage on tested catalogs.
Cross-vendor vocabulary cache	Production-ready. Reuses mappings across catalogs to cut cost over time.
Resume after crash, re-run cheaply	Production-ready. All intermediate state lives on disk.
Merging multiple catalogs into one buy-sheet	Beta. Wired end-to-end; merge logic only validated on synthetic test cases so far. Conflict resolution needs more real-world testing.
Photo embedding on multi-catalog runs	Out of scope. Disabled by design until the matcher handles cross-catalog cases.
PPTX vendor decks	Not supported. Convert externally first.

Operating model

Running a catalog

A buyer drops a vendor PDF into the system and runs one command. The pipeline does the rest — render, classify, extract, verify, map, build — and produces a filled BUYSHEET file. Anything uncertain is yellow-flagged.

Cost shape

Roughly \$3–\$5 in model API spend per typical 30-page catalog, almost entirely driven by the per-page extraction calls. The vocabulary cache is shared across vendors, so by the third catalog those lookups are nearly free. Re-running a catalog (for any reason) costs essentially nothing because every step's output is cached on disk.

Failure recovery

Every step is idempotent and resumable. If a run is interrupted, restarting picks up where it left off. There's a force flag to redo any step deliberately.

What we'd want next

- **Harden multi-catalog merging** with real vendor data so we can trust the conflict-resolution path.
- **Native PPTX adapter** so vendors who send slide decks don't need pre-conversion.
- **Auto-priority advisor** — a cheap probe that ranks which source to trust per field when catalogs disagree.